



ICT Training Center

Il tuo partner per la Formazione e la Trasformazione digitale della tua azienda



SPRING AI

GENERATIVE ARTIFICIAL INTELLIGENCE CON JAVA

Simone Scannapieco

Corso avanzato per Venis S.p.A, Venezia, Italia

Novembre 2025



SPEECH-TO-TEXT

- ➔ Processo di produzione testuale da audio del parlato
- ➔ Compatibilità Spring AI (al 11/2025)
 - ➔ OpenAI API
 - ➔ Azure OpenAI
 - ➔ OpenAI-compatible API (limitata)

Dipendenze applicativo

```
...  
<dependency>  
  <groupId>org.springframework.boot</groupId>  
  <artifactId>spring-boot-starter-web</artifactId>  
</dependency>  
<dependency>  
  <groupId>org.springframework.ai</groupId>  
  <artifactId>spring-ai-starter-model-openai</artifactId>  
</dependency>  
...
```

Configurazione applicativo

```
spring:  
  application:  
    name: demo  
  ai:  
    openai:  
      api-key: ${OPENAI_API_KEY}
```

Controller di trascrizione

```
@RestController
@RequestMapping("/api")
public class AudioController {

    private final OpenAiAudioTranscriptionModel openAiAudioTranscriptionModel;

    AudioController(OpenAiAudioTranscriptionModel transcriptionModel) {
        this.openAiAudioTranscriptionModel = transcriptionModel;
    }

    @GetMapping("/transcribe")
    String transcribe(@Value("classpath:filename.mp3") Resource audioFile) {
        return openAiAudioTranscriptionModel.call(audioFile);
    }
}
```

- ➔ `OpenAiAudioTranscriptionModel` implementazione di `AudioTranscriptionModel`
- ➔ Impostazioni di *default*
 - ➔ Modello Whisper 1
 - ➔ *Response format* in JSON
 - ➔ Temperatura 0
- ➔ Formati supportati: mp3, mp4, mpeg, mpga, m4a, wav, webm (documentazione classe `AudioTranscriptionPrompt`)
- ⚠ Metodo `call` per allinearsi all'astrazione LLM in Spring AI

Controller di trascrizione

```
@RestController
@RequestMapping("/api")
public class AudioController {

    ...

    @GetMapping("/transcribe-options")
    String transcribeWithOptions(@Value("classpath:filename.mp3") Resource audioFile) {
        var audioTranscriptionResponse = openAiAudioTranscriptionModel.call(new AudioTranscriptionPrompt(
            audioFile, OpenAiAudioTranscriptionOptions.builder()
                .prompt("Topics")
                .language("it")
                .temperature(0.5f)
                .responseFormat(OpenAiAudioApi.TranscriptResponseFormat.VTT).build()));
        return audioTranscriptionResponse.getResult().getOutput();
    }
}
```

- ➔ **AudioTranscriptionPrompt** incapsula la risorsa audio e le opzioni di trascrizione (**OpenAiAudioTranscriptionOptions**)
 - ➔ **.prompt** fornisce al LLM contesto riguardo gli argomenti trattati nel *file* audio
 - ➔ **.language** evita al LLM l'attività di *language guessing*
 - ➔ **.responseFormat** per calibrare il formato di *output* al contesto applicativo di utilizzo
- ⚠ con `call(audioFile)`, Spring AI si occupa di creare un *bean* **OpenAiAudioTranscriptionOptions** con i valori di *default*

Configurazione applicativo

```
spring:
  application:
    name: demo
  ai:
    openai:
      api-key: ${OPENAI_API_KEY}
      audio:
        transcription:
          model: whisper-1          # whisper-1, gpt-4o-transcribe, gpt-4o-mini-transcribe
          response-format: json    # json, text, srt, verbose_json, vtt
          prompt: Topics           # Should match the audio language
          language: it             # ISO-639-1 format to improve accuracy and latency
          temperature: 0          # [0, 1]. 0 means automatically adjusting temperature
          ...
```


- ⚠ Ollama non supportato
- ⚠ Gemini API supportato con restrizioni
 - ➔ Non possibile utilizzare le astrazioni `OpenAiAudioTranscriptionModel` e `AudioTranscriptionPrompt`
 - ➔ Configurazione statica non ha effetto
 - ➔ Sfruttare modelli con caratteristiche intrinseche STT...
 - ➔ ...come Gemini 2.5!

Upload file audio su cloud temporaneo Google

```
AUDIO_PATH="path/to/sample.mp3"
MIME_TYPE=$(file -b --mime-type "${AUDIO_PATH}")
NUM_BYTES=$(wc -c < "${AUDIO_PATH}")
DISPLAY_NAME=AUDIO

tmp_header_file=upload-header.tmp

# Initial resumable request defining metadata.
# The upload url is in the response headers dump them to a file.
curl "https://generativelanguage.googleapis.com/upload/v1beta/files" \
  -H "x-goog-api-key: $GEMINI_API_KEY" \
  -D upload-header.tmp \
  -H "X-Goog-Upload-Protocol: resumable" \
  -H "X-Goog-Upload-Command: start" \
  -H "X-Goog-Upload-Header-Content-Length: ${NUM_BYTES}" \
  -H "X-Goog-Upload-Header-Content-Type: ${MIME_TYPE}" \
  -H "Content-Type: application/json" \
  -d '{"file': {'display_name': '${DISPLAY_NAME}'}}" 2> /dev/null

upload_url=$(grep -i "x-goog-upload-url: " "${tmp_header_file}" | cut -d" " -f2 | tr -d "\r")
rm "${tmp_header_file}"

# Upload the actual bytes.
curl "${upload_url}" \
  -H "Content-Length: ${NUM_BYTES}" \
  -H "X-Goog-Upload-Offset: 0" \
  -H "X-Goog-Upload-Command: upload, finalize" \
  --data-binary "@${AUDIO_PATH}" 2> /dev/null > file_info.json

file_uri=$(jq ".file.uri" file_info.json)
echo file_uri=$file_uri
```

Chiamata Gemini API *generateContent*

```
# Now generate content using that file
curl "https://generativelanguage.googleapis.com/v1beta/models/gemini-2.5-flash:generateContent" \
-H "x-goog-api-key: $GEMINI_API_KEY" \
-H 'Content-Type: application/json' \
-X POST \
-d '{
  "contents": [{
    "parts": [
      {
        "text": "Describe this audio clip",
        "file_data": {
          "mime_type": "${MIME_TYPE}",
          "file_uri": "$file_uri"
        }
      }
    ]
  }]
}' 2> /dev/null > response.json

cat response.json
echo

jq ".candidates[].content.parts[].text" response.json
```

➔ Due strade percorribili

- 1 Utilizzo Spring Web puro (RestTemplate, ResponseEntity, HttpEntity, ...)
- 2 Utilizzo Spring AI (STT attraverso ChatClient)

- ➔ Servizio *speech-to-text* Gemini
 - 1 Creazione interfaccia ed implementazione del servizio
 - 2 Modifica controllore MVC
 - 3 Test delle funzionalità con Postman/Insomnia

Interfaccia servizio

```
package it.venis.ai.spring.demo.services;  
  
import org.springframework.core.io.Resource;  
  
import it.venis.ai.spring.demo.model.Answer;  
  
public interface MultiModalityService {  
  
    Answer getTranscriptionFromAudioFile(Resource audioFile);  
  
}
```

Implementazione servizio - I

```
package it.venis.ai.spring.demo.services;

...

@Service
public class MultiModalityServiceImpl implements MultiModalityService {

    private final ChatClient geminiChatClient;

    public MultiModalityServiceImpl(@Qualifier("geminiChatClient") ChatClient geminiChatClient) {

        this.geminiChatClient = geminiChatClient;
    }

    @Override
    public Answer getTranscriptionFromAudioFile(Resource audioFile) {

        String contentType = getContentType(audioFile);

        Media audioMedia = new Media(
            MimeTypeUtils.parseMimeType(contentType), audioFile);

        return new Answer(this.geminiChatClient.prompt()
            .user(u -> u.text("Trascrivi il seguente file audio: ").media(audioMedia))
            .call()
            .content());
    }

    ...
}
```

Implementazione servizio - II

```
...  
  
private String getContentType(Resource multimediaFile) {  
    String contentType = URLConnection.guessContentTypeFromName(multimediaFile.getFilename());  
  
    if (contentType == null || contentType.equals("application/octet-stream")) {  
        String fileName = multimediaFile.getFilename();  
        if (fileName != null) {  
            String extension = fileName.substring(fileName.lastIndexOf('.') + 1).toLowerCase();  
            contentType = switch (extension) {  
                case "mp3" -> "audio/mpeg";  
                case "wav" -> "audio/wav";  
                case "m4a" -> "audio/mp4";  
                case "ogg" -> "audio/ogg";  
                case "flac" -> "audio/flac";  
                default -> "media/unknown";  
            };  
        } else {  
            contentType = "media/unknown";  
        }  
    }  
  
    return contentType;  
}  
  
}
```

Implementazione controllore REST

```
package it.venis.ai.spring.demo.controllers;

...

@RestController
@RequestMapping("/gemini/multi-modality")
public class GeminiMultimodalityController {

    private final MultiModalityService multiModalityService;

    public GeminiMultimodalityController(MultiModalityService multiModalityService) {

        this.multiModalityService = multiModalityService;
    }

    /**
     * Endpoint per la trascrizione di file audio in testo (Speech-to-Text).
     * Utilizza un file audio predefinito dal classpath e ne estrae la trascrizione testuale
     * tramite i servizi di riconoscimento vocale di Gemini.
     *
     * @param audioFile risorsa audio dal classpath (file WAV predefinito)
     * @return Answer oggetto contenente la trascrizione del file audio
     */
    @PostMapping("/stt")
    public Answer getTranscriptionFromAudioFile(@Value("classpath:Venis_descrizione_azienza.wav") Resource audioFile) {

        return this.multiModalityService.getTranscriptionFromAudioFile(audioFile);
    }
}
```


<https://github.com/simonescannapieco/spring-ai-advanced-dgroove-venis-code.git>
Branch: 16-spring-ai-gemini-ollama-multimodality-stt